

Code-Layout, Readability and Reusability

Date	15 March 2024
Team ID	B Siva Krishna Santhosh Gowthami Botsa Jaya Sarvani Vengalasetty Pilipe Praveen Vaishnavi P
Project Name	EcoTrack - AI-Powered Carbon Footprint Tracker
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the EcoTrack codebase in terms of its structure, readability, and reusability. The project follows a React.js (Vite) frontend and Flask (Python) backend architecture with a dual-mode database layer (Firestore / SQLite fallback). Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	React components and Flask routes follow consistent 2/4-space indentation.
2	Proper File Structure	Files and folders are logically organized	Yes	Clearly split into /frontend and /backend, with component and route separation.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names like dailyFootprint, habitLogs, carbonBudget are self-explanatory.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. calculateEmissions(), logHabit(), generateSuggestions() clearly state intent.
5	Code Comments	Inline and block comments explain logic	Partial	Core calculation and API integration logic is commented; some UI components lack comments.
6	Modular Design	Code is split into reusable functions/modules	Yes	Separate modules for auth, habit logging, emission calculation, and AI suggestions.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Minor duplication exists between travel/food/energy form handlers; can be refactored into shared components.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Backend includes fallback logic (local SQLite, static emission factors) when external services/APIs are unavailable.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	HabitForm (multi-step logger)	React.js (JSX)	Reused across Travel, Food, and Energy logging steps	High
2	CarbonGauge component	React.js + Chart.js	Reused on Dashboard and History pages to display emission scores	High
3	API service layer (Axios instance)	JavaScript (Axios)	Reused by all frontend pages for authenticated backend calls	High
4	Emission Calculator module	Python (Flask)	Reused by travel, food, and energy calculation endpoints	High
5	AuthMiddleware / token verification	Python (Flask) / Firebase Auth	Reused across all protected backend API routes	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear separation between frontend and backend; consistent formatting throughout.
Readability	4	Descriptive naming conventions make the logic easy to follow.
Reusability	4	Core modules (habit form, gauge, calculator) are well-factored for reuse.
Documentation / Comments	3	Key logic is commented, but some frontend components could use more inline documentation.
Overall Score	4	A well-structured, maintainable full-stack project with minor room for refactoring and documentation improvements.